

# AI Applications Lecture 7

## Embedding Layer and Distances/Similarities

---

SUZUKI, Atsushi<sup>a</sup>

---

<sup>a</sup>I thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

# Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can a neural network represent the diversity of words? - Embedding
4. Can we use representations in a NN? - Distance and similarities
5. How can we input order information? - Positional encoding
6. Summary

## Review of the previous lecture

In the previous lecture, we focused on the **token generator**.

## Review of the previous lecture

In the previous lecture, we focused on the **token generator**.

**Sampling** maps the **continuous output (the probability mass)** of a **NN** to a **discrete object (token)**.

## Review of the previous lecture

In the previous lecture, we focused on the **token generator**.

**Sampling** maps the **continuous output (the probability mass)** of a **NN** to a **discrete object (token)**.

In this lecture, we will look at its dual: converting a discrete input into a continuous object at the input phase.

## Review of the previous lecture

In the previous lecture, we focused on the **token generator**.

**Sampling** maps the **continuous output (the probability mass) of a NN** to a **discrete object (token)**.

In this lecture, we will look at its dual: converting a discrete input into a continuous object at the input phase.

The **embedding layer** is responsible for the discrete-continuous conversion.

## Learning outcomes for this lecture

By the end of this lecture, you should be able to:

- Explain what an **embedding layer** does.

## Learning outcomes for this lecture

By the end of this lecture, you should be able to:

- Explain what an **embedding layer** does.
- Calculate the **distances and similarities** (Euclidean distance, standard inner product, cosine similarity) between token representations (vectors) obtained through embedding.

# Outline

## 1. Introduction

## 2. Preliminaries: Mathematical Notations

## 3. How can a neural network represent the diversity of words? - Embedding

## 4. Can we use representations in a NN? - Distance and similarities

## 5. How can we input order information? - Positional encoding

## 6. Summary

# Outline

1. Introduction

## **2. Preliminaries: Mathematical Notations**

3. How can a neural network represent the diversity of words? - Embedding

4. Can we use representations in a NN? - Distance and similarities

5. How can we input order information? - Positional encoding

6. Summary

# Preliminaries: mathematical notations

## Set & Function:

- Sets:  $\mathcal{A}$
- Membership:  $x \in \mathcal{A}$
- Integer range:  $[1, k]_{\mathbb{Z}} := \{1, \dots, k\}$
- Real numbers:  $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Function:  $f : \mathcal{X} \rightarrow \mathcal{Y}$

## Vector: Denoted by $\boldsymbol{v}$ .

- A column of numbers,  $\boldsymbol{v} \in \mathbb{R}^n$ .
- $i$ -th element is  $v_i$ .
- Standard inner product:  
 $\langle \boldsymbol{u}, \boldsymbol{v} \rangle = \boldsymbol{u}^{\top} \boldsymbol{v}$

# Preliminaries: mathematical notations

**Sequence:** Denoted by  $\mathbf{a} = (a_1, a_2, \dots)$ .

- A function  $\mathbf{a} : [1, n]_{\mathbb{Z}} \rightarrow \mathcal{A}$ .
- Length is denoted by  $|\mathbf{a}|$ .

**Matrix:** Denoted by  $\mathbf{A}$ .

- $m \times n$  matrix:  $\mathbf{A} \in \mathbb{R}^{m,n}$ .
- $(i, j)$ -th element is  $a_{i,j}$ .

**Tensor:** Denoted by  $\underline{\mathbf{A}}$ .

- Simply a multi-dimensional array.
- Vector  $\rightarrow$  1st-order, Matrix  $\rightarrow$  2nd-order.

# Outline

1. Introduction

## **2. Preliminaries: Mathematical Notations**

3. How can a neural network represent the diversity of words? - Embedding

4. Can we use representations in a NN? - Distance and similarities

5. How can we input order information? - Positional encoding

6. Summary

# Outline

1. Introduction

2. Preliminaries: Mathematical Notations

**3. How can a neural network represent the diversity of words? - Embedding**

4. Can we use representations in a NN? - Distance and similarities

5. How can we input order information? - Positional encoding

6. Summary

**Question:** Words are not uniform. The one-hot encoding doesn't seem to represent such non-uniformity. How

**Question:** Words are not uniform. The one-hot encoding doesn't seem to represent such non-uniformity. How can the neural network represent each word's characteristic?

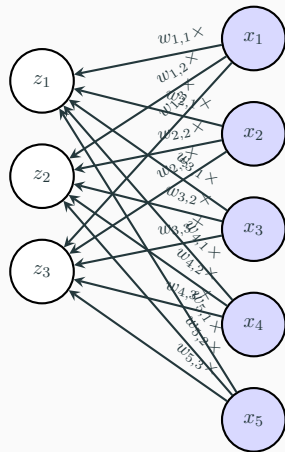
## Symmetry and fully-connected layer

When the **input is one-hot**, we have no choice but to treat **each input node equally**. The natural connection to the next set of nodes is a **fully-connected** layer.

## Symmetry and fully-connected layer

When the **input is one-hot**, we have no choice but to treat **each input node equally**. The natural connection to the next set of nodes is a **fully-connected** layer.

This part of the network is called an **embedding layer**.



**Figure 1:** A fully-connected layer.

# Matrix representation and equivalence to linear regression

A fully-connected layer is a **linear map** and can be written using a matrix

$\mathbf{W} \in \mathbb{R}^{d_{\text{emb}}, d_{\text{in}}}$ :

$$\mathbf{z} := \mathbf{W} \mathbf{x} \quad (\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}, \mathbf{z} \in \mathbb{R}^{d_{\text{emb}}}). \quad (1)$$

## Active edges for a one-hot input

When the input is a one-hot vector,  
 $x = e_i$ , something interesting happens.

## Active edges for a one-hot input

When the input is a one-hot vector,  $x = e_i$ , something interesting happens. The multiplication  $W e_i$  simply selects the  $i$ -th column of the matrix  $W$ .

$$W e_i = w_i \quad (2)$$

(where  $w_i$  is the  $i$ -th column of  $W$ ).

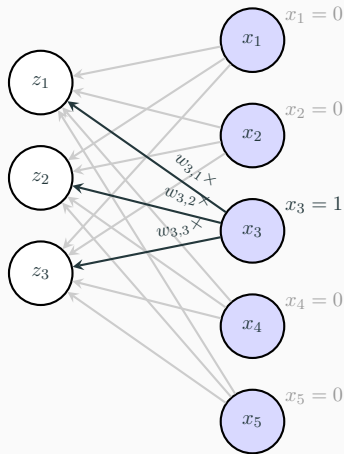


Figure 2: Active edges for input  $e_3$ . 14/63

# Equivalent understanding of embedding

This means:

- "Feeding a one-hot vector  $e_i$  into the first fully-connected layer"...

# Equivalent understanding of embedding

This means:

- "Feeding a one-hot vector  $e_i$  into the first fully-connected layer"...

...is **equivalent** to...

- "Directly looking up the column vector  $w_i$  in the weight matrix".

# Equivalent understanding of embedding

This means:

- "Feeding a one-hot vector  $e_i$  into the first fully-connected layer"...

...is **equivalent** to...

- "Directly looking up the column vector  $w_i$  in the weight matrix".

The column vector  $w_i$  is a **learnable parameter** vector that represents token  $i$ .

# Equivalent understanding of embedding

## Definition (Embedding)

The map defined by each column  $\boldsymbol{w}_i$  of the matrix  $\boldsymbol{W} \in \mathbb{R}^{d_{\text{emb}}, d_{\text{in}}}$ ,

$$\iota : i \in \{1, \dots, d_{\text{in}}\} \mapsto \boldsymbol{w}_i \in \mathbb{R}^{d_{\text{emb}}} \quad (3)$$

is called an **embedding** or **representation** [1, 2, 3].

# Equivalent understanding of embedding

## Remark

It is customary to call  $w_i$  the **representation** of the  $i$ -th token or the **embedding** of the  $i$ -th token.

# Equivalent understanding of embedding

## Remark

It is customary to call  $w_i$  the **representation** of the  $i$ -th token or the **embedding** of the  $i$ -th token.

Mathematically, an "embedding" is an **injective map that preserves structure**.

# Equivalent understanding of embedding

## Remark

It is customary to call  $w_i$  the **representation** of the  $i$ -th token or the **embedding** of the  $i$ -th token.

Mathematically, an "embedding" is an **injective map that preserves structure**.

This name is used with the expectation that the relationships between tokens will be **reflected** in the relationships between their representations in the vector space.

## Vocabulary extension (adding columns)

How do we add new tokens to our vocabulary?

## Vocabulary extension (adding columns)

How do we add new tokens to our vocabulary?

- We increase the dimension of the one-hot vectors (i.e., increase  $d_{\text{in}}$ ).

## Vocabulary extension (adding columns)

How do we add new tokens to our vocabulary?

- We increase the dimension of the one-hot vectors (i.e., increase  $d_{\text{in}}$ ).
- In the matrix representation, we simply **add new column vectors to  $W$** .

## Vocabulary extension (adding columns)

How do we add new tokens to our vocabulary?

- We increase the dimension of the one-hot vectors (i.e., increase  $d_{\text{in}}$ ).
- In the matrix representation, we simply **add new column vectors to  $W$** .
- It is possible to **train only the new columns**, or fine-tune the entire matrix.

## Embedding - answer

**Question:** How can the neural network represent each word's characteristic?

**Answer:**

## Embedding - answer

**Question:** How can the neural network represent each word's characteristic?

**Answer:** The weights of the edges connecting to the input nodes implicitly represent the characteristics of each token.

# Outline

1. Introduction

2. Preliminaries: Mathematical Notations

**3. How can a neural network represent the diversity of words? - Embedding**

4. Can we use representations in a NN? - Distance and similarities

5. How can we input order information? - Positional encoding

6. Summary

# Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can a neural network represent the diversity of words? - Embedding
- 4. Can we use representations in a NN? - Distance and similarities**
5. How can we input order information? - Positional encoding
6. Summary

**Question:** An embedding layer in a NN convert each token to a vector. Can

**Question:** An embedding layer in a NN convert each token to a vector. Can we interpret them?

## Distances and similarities in the embedding space

## Distances and similarities in the embedding space

If the embedding reflects semantic information, calculating the **relationships between vectors** can have practical applications [2, 1].

## Distances and similarities in the embedding space

If the embedding reflects semantic information, calculating the **relationships between vectors** can have practical applications [2, 1].

Here, we will strictly define the most fundamental measures.

# Distances and similarities in the embedding space

## Definition (Euclidean Distance)

For  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{d_{\text{emb}}}$ , the **Euclidean distance**  $d_E(\mathbf{u}, \mathbf{v})$  is defined as:

$$d_E(\mathbf{u}, \mathbf{v}) := \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{(\mathbf{u} - \mathbf{v})^\top (\mathbf{u} - \mathbf{v})}. \quad (4)$$

## Distances and similarities in the embedding space

### Definition (Euclidean Distance)

For  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{d_{\text{emb}}}$ , the **Euclidean distance**  $d_E(\mathbf{u}, \mathbf{v})$  is defined as:

$$d_E(\mathbf{u}, \mathbf{v}) := \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{(\mathbf{u} - \mathbf{v})^\top (\mathbf{u} - \mathbf{v})}. \quad (4)$$

The **smaller** the distance, the more similar the concepts.

# Distances and similarities in the embedding space

## Definition (Standard Inner Product)

For  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{d_{\text{emb}}}$ , the **Standard Inner Product**  $\langle \mathbf{u}, \mathbf{v} \rangle$  is defined as:

$$\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{k=1}^{d_{\text{emb}}} u_k v_k. \quad (5)$$

# Distances and similarities in the embedding space

## Definition (Standard Inner Product)

For  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{d_{\text{emb}}}$ , the **Standard Inner Product**  $\langle \mathbf{u}, \mathbf{v} \rangle$  is defined as:

$$\langle \mathbf{u}, \mathbf{v} \rangle := \mathbf{u}^\top \mathbf{v} = \sum_{k=1}^{d_{\text{emb}}} u_k v_k. \quad (5)$$

The **larger** the inner product, the more similar the concepts. The Transformer architecture [5] uses this measure extensively.

# Distances and similarities in the embedding space

## Definition (Cosine Similarity)

For  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{d_{\text{emb}}} \setminus \{0\}$ , the **cosine similarity**  $\text{cos}(\mathbf{u}, \mathbf{v})$  is defined as:

$$\text{cos}(\mathbf{u}, \mathbf{v}) := \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}. \quad (6)$$

## Distances and similarities in the embedding space

### Definition (Cosine Similarity)

For  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{d_{\text{emb}}} \setminus \{0\}$ , the **cosine similarity**  $\text{cos}(\mathbf{u}, \mathbf{v})$  is defined as:

$$\text{cos}(\mathbf{u}, \mathbf{v}) := \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}. \quad (6)$$

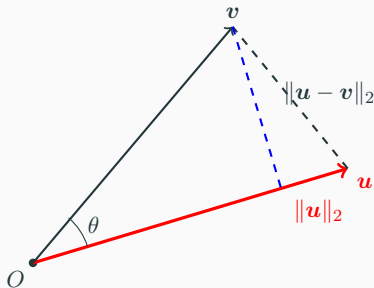
This is the cosine of the angle between the two vectors. The **larger** the similarity (closer to 1), the more similar the concepts.

## Distances and similarities in the embedding space

The following identity holds:

$$\begin{aligned}\|u - v\|_2^2 &= \|u\|_2^2 + \|v\|_2^2 - 2 \langle u, v \rangle \quad (7) \\ &= \|u\|_2^2 + \|v\|_2^2 \\ &\quad - 2 \|u\|_2 \|v\|_2 \cos \theta,\end{aligned}$$

where  $\theta$  is the angle between  $u$  and  $v$ .

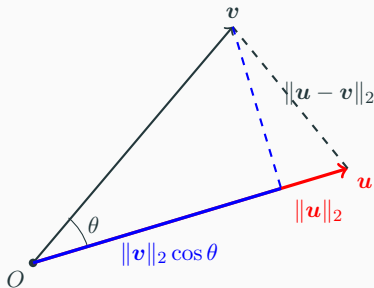


## Distances and similarities in the embedding space

The following identity holds:

$$\begin{aligned}\|u - v\|_2^2 &= \|u\|_2^2 + \|v\|_2^2 - 2 \langle u, v \rangle \quad (7) \\ &= \|u\|_2^2 + \|v\|_2^2 \\ &\quad - 2 \|u\|_2 \|v\|_2 \cos \theta,\end{aligned}$$

where  $\theta$  is the angle between  $u$  and  $v$ .



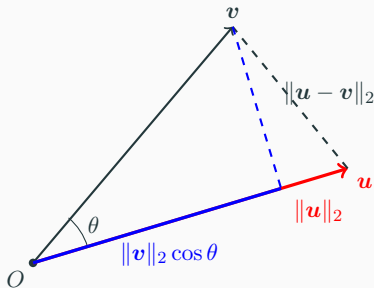
## Distances and similarities in the embedding space

The following identity holds:

$$\begin{aligned}\|u - v\|_2^2 &= \|u\|_2^2 + \|v\|_2^2 - 2 \langle u, v \rangle \quad (7) \\ &= \|u\|_2^2 + \|v\|_2^2 \\ &\quad - 2 \|u\|_2 \|v\|_2 \cos \theta,\end{aligned}$$

where  $\theta$  is the angle between  $u$  and  $v$ .

The Euclidean distance  $\|u - v\|_2^2$  is **decreasing** wrt  $\theta$  or  $\langle u, v \rangle$  if  $\|u\|_2$  and  $\|v\|_2$  are fixed.



## Examples and exercises

**Step 1: Calculate the difference vector**

$$u - v = \begin{bmatrix} 2 - 1 \\ -1 - 2 \\ 3 - (-1) \end{bmatrix} = \begin{bmatrix} 1 \\ -3 \\ 4 \end{bmatrix} \quad (8)$$

## Examples and exercises

**Step 2: Calculate the L2 norm of the difference**

$$\|u - v\|_2 = \sqrt{1^2 + (-3)^2 + 4^2} \tag{9}$$

$$= \sqrt{1 + 9 + 16} = \sqrt{26}. \tag{10}$$

The Euclidean distance is  $\sqrt{26}$ .

## Examples and exercises

### Exercise

Find the Euclidean distance  $d_E(\mathbf{a}, \mathbf{b})$  for  $\mathbf{a} = \begin{bmatrix} -1 \\ 4 \\ 2 \end{bmatrix}$  and  $\mathbf{b} = \begin{bmatrix} 3 \\ 0 \\ -2 \end{bmatrix}$ .

## Examples and exercises

### Answer

The difference vector is:

$$\mathbf{a} - \mathbf{b} = \begin{bmatrix} -1 - 3 \\ 4 - 0 \\ 2 - (-2) \end{bmatrix} = \begin{bmatrix} -4 \\ 4 \\ 4 \end{bmatrix} \quad (11)$$

## Examples and exercises

### Answer

The difference vector is:

$$\mathbf{a} - \mathbf{b} = \begin{bmatrix} -1 - 3 \\ 4 - 0 \\ 2 - (-2) \end{bmatrix} = \begin{bmatrix} -4 \\ 4 \\ 4 \end{bmatrix} \quad (11)$$

The Euclidean distance is:

$$\|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{(-4)^2 + 4^2 + 4^2} = \sqrt{16 + 16 + 16} = \sqrt{48} = 4\sqrt{3}. \quad (12)$$

## Examples and exercises

### Example (Inner Product)

For  $u = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}$  and  $v = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}$ , find the inner product.

## Examples and exercises

$$\langle u, v \rangle = 2 \cdot 1 + (-1) \cdot 2 + 3 \cdot (-1) \quad (13)$$

## Examples and exercises

$$\langle \mathbf{u}, \mathbf{v} \rangle = 2 \cdot 1 + (-1) \cdot 2 + 3 \cdot (-1) \quad (13)$$

$$= 2 - 2 - 3 = -3. \quad (14)$$

The inner product is  $-3$ .

## Examples and exercises

### Exercise

Find the inner product  $\langle a, b \rangle$  for  $a = \begin{bmatrix} -1 \\ 4 \\ 2 \end{bmatrix}$  and  $b = \begin{bmatrix} 3 \\ 0 \\ -2 \end{bmatrix}$ .

## Examples and exercises

**Answer**

$$\langle a, b \rangle = (-1) \cdot 3 + 4 \cdot 0 + 2 \cdot (-2) \quad (15)$$

## Examples and exercises

**Answer**

$$\langle a, b \rangle = (-1) \cdot 3 + 4 \cdot 0 + 2 \cdot (-2) \quad (15)$$

$$= -3 + 0 - 4 = -7. \quad (16)$$

## Examples and exercises

### Example (Cosine Similarity)

For  $u = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}$  and  $v = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}$ , find the cosine similarity.

# Examples and exercises

## Step 1: Reuse the inner product

$$\langle u, v \rangle = -3 \quad (\text{from previous example}) \quad (17)$$

## Examples and exercises

### Step 2: Calculate the norms

$$\|\mathbf{u}\|_2 = \sqrt{2^2 + (-1)^2 + 3^2} = \sqrt{4 + 1 + 9} = \sqrt{14} \quad (18)$$

$$\|\mathbf{v}\|_2 = \sqrt{1^2 + 2^2 + (-1)^2} = \sqrt{1 + 4 + 1} = \sqrt{6} \quad (19)$$

## Examples and exercises

**Step 3: Divide inner product by product of norms**

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{-3}{\sqrt{14} \sqrt{6}} = \frac{-3}{\sqrt{84}} = \frac{-3}{2\sqrt{21}}. \quad (20)$$

## Examples and exercises

### Exercise

Find the cosine similarity for  $a = \begin{bmatrix} -1 \\ 4 \\ 2 \end{bmatrix}$  and  $b = \begin{bmatrix} 3 \\ 0 \\ -2 \end{bmatrix}$ .

## Examples and exercises

### Answer

$$\langle \mathbf{a}, \mathbf{b} \rangle = -7 \quad (\text{from previous exercise}) \quad (21)$$

$$\|\mathbf{a}\|_2 = \sqrt{(-1)^2 + 4^2 + 2^2} = \sqrt{1 + 16 + 4} = \sqrt{21} \quad (22)$$

$$\|\mathbf{b}\|_2 = \sqrt{3^2 + 0^2 + (-2)^2} = \sqrt{9 + 0 + 4} = \sqrt{13} \quad (23)$$

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{-7}{\sqrt{21} \sqrt{13}} = \frac{-7}{\sqrt{273}}. \quad (24)$$

## Distance and similarity - answer

**Question:** An embedding layer in a NN convert each token to a vector. Can we interpret them?

**Answer:** We can quantify the distance or similarity of token representation vectors in the NN.

# Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can a neural network represent the diversity of words? - Embedding
- 4. Can we use representations in a NN? - Distance and similarities**
5. How can we input order information? - Positional encoding
6. Summary

# Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can a neural network represent the diversity of words? - Embedding
4. Can we use representations in a NN? - Distance and similarities
- 5. How can we input order information? - Positional encoding**
6. Summary

**Question:** The meaning of a token can depend strongly on its position in a sequence. How can we input **order information** into a Transformer?

## Why do we need positional information?

For example, a Transformer applies essentially the **same structure and parameters at every token position**.

# Why do we need positional information?

For example, a Transformer applies essentially the **same structure and parameters at every token position**.

This has important advantages:

# Why do we need positional information?

For example, a Transformer applies essentially the **same structure and parameters at every token position**.

This has important advantages:

- The same parameters can be reused at every position, giving good **parameter efficiency**.

# Why do we need positional information?

For example, a Transformer applies essentially the **same structure and parameters at every token position**.

This has important advantages:

- The same parameters can be reused at every position, giving good **parameter efficiency**.
- The same architecture can be applied directly to **variable-length sequences**.

# Why do we need positional information?

For example, a Transformer applies essentially the **same structure and parameters at every token position**.

This has important advantages:

- The same parameters can be reused at every position, giving good **parameter efficiency**.
- The same architecture can be applied directly to **variable-length sequences**.

However, this symmetry also creates a problem: without positional information, the network does not intrinsically know **where** each token appears.

## Permutation equivariance without positional information

Let  $\mathbf{t} = (t^{(1)}, t^{(2)}, \dots, t^{(L)})$  be an input sequence.

## Permutation equivariance without positional information

Let  $\mathbf{t} = (t^{(1)}, t^{(2)}, \dots, t^{(L)})$  be an input sequence.

A Transformer constructed only from position-wise operations and self-attention is **permutation equivariant**:

$$f_{\theta}(\sigma(\mathbf{t})) = \sigma(f_{\theta}(\mathbf{t})), \quad \sigma \text{ is an arbitrary permutation..} \quad (25)$$

## Permutation equivariance without positional information

Let  $\mathbf{t} = (t^{(1)}, t^{(2)}, \dots, t^{(L)})$  be an input sequence.

A Transformer constructed only from position-wise operations and self-attention is **permutation equivariant**:

$$f_{\theta}(\sigma(\mathbf{t})) = \sigma(f_{\theta}(\mathbf{t})), \quad \sigma \text{ is an arbitrary permutation..} \quad (25)$$

Thus, permuting the input tokens merely permutes the corresponding outputs in the same way.

## Why is permutation equivariance a problem?

Consider the two sequences

(wine, red)      and      (red, wine).      (26)

## Why is permutation equivariance a problem?

Consider the two sequences

(wine, red)      and      (red, wine).      (26)

If the network has no information about token positions, exchanging the two tokens can only exchange their corresponding output representations.

# Why is permutation equivariance a problem?

Consider the two sequences

(wine, red)      and      (red, wine).      (26)

If the network has no information about token positions, exchanging the two tokens can only exchange their corresponding output representations.

However, in natural language, **changing word order can substantially change the meaning.**

# Why is permutation equivariance a problem?

Consider the two sequences

$$(\text{wine}, \text{red}) \quad \text{and} \quad (\text{red}, \text{wine}). \quad (26)$$

If the network has no information about token positions, exchanging the two tokens can only exchange their corresponding output representations.

However, in natural language, **changing word order can substantially change the meaning.**

Therefore, token identity alone is not sufficient. We also want to tell the network **where each token occurs.**

## Adding position to the input

**The idea of positional embedding:** Instead of regarding the input only as

$$\mathbf{t} = \left( t^{(1)}, t^{(2)}, \dots, t^{(L)} \right), \quad (27)$$

we regard each token together with its position:

$$\left( (1, t^{(1)}), (2, t^{(2)}), \dots, (L, t^{(L)}) \right). \quad (28)$$

## Adding position to the input

**The idea of positional embedding:** Instead of regarding the input only as

$$\mathbf{t} = \left( t^{(1)}, t^{(2)}, \dots, t^{(L)} \right), \quad (27)$$

we regard each token together with its position:

$$\left( (1, t^{(1)}), (2, t^{(2)}), \dots, (L, t^{(L)}) \right). \quad (28)$$

The neural network can therefore use both:

- **what**

## Adding position to the input

**The idea of positional embedding:** Instead of regarding the input only as

$$\mathbf{t} = \left( t^{(1)}, t^{(2)}, \dots, t^{(L)} \right), \quad (27)$$

we regard each token together with its position:

$$\left( (1, t^{(1)}), (2, t^{(2)}), \dots, (L, t^{(L)}) \right). \quad (28)$$

The neural network can therefore use both:

- **what** the token is, and
- **where**

## Adding position to the input

**The idea of positional embedding:** Instead of regarding the input only as

$$\mathbf{t} = \left( t^{(1)}, t^{(2)}, \dots, t^{(L)} \right), \quad (27)$$

we regard each token together with its position:

$$\left( (1, t^{(1)}), (2, t^{(2)}), \dots, (L, t^{(L)}) \right). \quad (28)$$

The neural network can therefore use both:

- **what** the token is, and
- **where** the token appears.

## Adding position to the input

**The idea of positional embedding:** Instead of regarding the input only as

$$\mathbf{t} = \left( t^{(1)}, t^{(2)}, \dots, t^{(L)} \right), \quad (27)$$

we regard each token together with its position:

$$\left( (1, t^{(1)}), (2, t^{(2)}), \dots, (L, t^{(L)}) \right). \quad (28)$$

The neural network can therefore use both:

- **what** the token is, and
- **where** the token appears.

Mechanisms for supplying such positional information are called **positional embedding**.

# Positional embedding

## Definition (Positional Embedding)

A vector-valued map

$$\mathbf{p} : i \in \mathbb{Z}_{>0} \longmapsto \mathbf{p}_i \in \mathbb{R}^{d_{\text{pos}}} \quad (29)$$

that converts the position  $i$  into a vector representation used by a neural network is called a **positional embedding**.

# Positional embedding

## Definition (Positional Embedding)

A vector-valued map

$$\mathbf{p} : i \in \mathbb{Z}_{>0} \mapsto \mathbf{p}_i \in \mathbb{R}^{d_{\text{pos}}} \quad (29)$$

that converts the position  $i$  into a vector representation used by a neural network is called a **positional embedding**.

Thus, a positional embedding represents

$$\text{position} \longrightarrow \text{vector}. \quad (30)$$

# Positional embedding

## Definition (Positional Embedding)

A vector-valued map

$$\mathbf{p} : i \in \mathbb{Z}_{>0} \longmapsto \mathbf{p}_i \in \mathbb{R}^{d_{\text{pos}}} \quad (29)$$

that converts the position  $i$  into a vector representation used by a neural network is called a **positional embedding**.

Thus, a positional embedding represents

$$\text{position} \longrightarrow \text{vector}. \quad (30)$$

The vector might be regarded as a matrix inside the NN.

## Example 1: sinusoidal positional embedding

The original Transformer uses a deterministic positional embedding based on sine and cosine functions [5].

## Example 1: sinusoidal positional embedding

The original Transformer uses a deterministic positional embedding based on sine and cosine functions [5].

Assume that the token representation dimension  $d_{\text{emb}}$  is even. For

$$k \in \left[0, \frac{d_{\text{emb}}}{2} - 1\right]_{\mathbb{Z}}, \quad (31)$$

define

$$(\mathbf{p}_i)_{2k+1} := \sin \left( \frac{i}{10000^{2k/d_{\text{emb}}}} \right), \quad (32)$$

$$(\mathbf{p}_i)_{2k+2} := \cos \left( \frac{i}{10000^{2k/d_{\text{emb}}}} \right). \quad (33)$$

## Sinusoidal positional embedding

The positional embedding is therefore the function

$$i \longmapsto \mathbf{p}_i = \begin{bmatrix} \sin(i) \\ \cos(i) \\ \sin(i/10000^{2/d_{\text{emb}}}) \\ \cos(i/10000^{2/d_{\text{emb}}}) \\ \vdots \end{bmatrix} \in \mathbb{R}^{d_{\text{emb}}}. \quad (34)$$

## Sinusoidal positional embedding

The positional embedding is therefore the function

$$i \longmapsto \mathbf{p}_i = \begin{bmatrix} \sin(i) \\ \cos(i) \\ \sin(i/10000^{2/d_{\text{emb}}}) \\ \cos(i/10000^{2/d_{\text{emb}}}) \\ \vdots \end{bmatrix} \in \mathbb{R}^{d_{\text{emb}}}. \quad (34)$$

Different coordinates oscillate at different frequencies.

## Sinusoidal positional embedding

The positional embedding is therefore the function

$$i \longmapsto \mathbf{p}_i = \begin{bmatrix} \sin(i) \\ \cos(i) \\ \sin(i/10000^{2/d_{\text{emb}}}) \\ \cos(i/10000^{2/d_{\text{emb}}}) \\ \vdots \end{bmatrix} \in \mathbb{R}^{d_{\text{emb}}}. \quad (34)$$

Different coordinates oscillate at different frequencies.

The original Transformer supplies position information by adding this vector to the token embedding:

$$\mathbf{x}^{(i)} = \iota(t^{(i)}) + \mathbf{p}_i. \quad (35)$$

## Example 2: Rotary Position Embedding (RoPE)

Another widely used positional encoding is the **Rotary Position Embedding (RoPE)** [4].

## Example 2: Rotary Position Embedding (RoPE)

Another widely used positional encoding is the **Rotary Position Embedding (RoPE)** [4].

RoPE maps a position to a **rotation matrix**:  $i \longmapsto R_i$ .

## Example 2: Rotary Position Embedding (RoPE)

Another widely used positional encoding is the **Rotary Position Embedding (RoPE)** [4].

RoPE maps a position to a **rotation matrix**:  $i \longmapsto R_i$ .

Thus, under the vector-valued definition given above, RoPE is more precisely a **positional encoding mechanism** rather than an ordinary positional embedding vector.

## Two-dimensional rotation

First define the two-dimensional rotation matrix

$$\text{Rot}(\phi) := \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}. \quad (36)$$

## Two-dimensional rotation

First define the two-dimensional rotation matrix

$$\text{Rot}(\phi) := \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}. \quad (36)$$

It satisfies

$$\text{Rot}(\phi)^T = \text{Rot}(-\phi), \quad (37)$$

$$\text{Rot}(\phi) \text{Rot}(\psi) = \text{Rot}(\phi + \psi). \quad (38)$$

## Two-dimensional rotation

First define the two-dimensional rotation matrix

$$\text{Rot}(\phi) := \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}. \quad (36)$$

It satisfies

$$\text{Rot}(\phi)^T = \text{Rot}(-\phi), \quad (37)$$

$$\text{Rot}(\phi) \text{Rot}(\psi) = \text{Rot}(\phi + \psi). \quad (38)$$

Consequently,

$$\text{Rot}(\phi)^T \text{Rot}(\psi) = \text{Rot}(\psi - \phi). \quad (39)$$

## RoPE as a function from position to a matrix

Let the dimension be  $d_{\text{rot}} = 2m$  and choose frequencies

$$\theta_k := 10000^{-2(k-1)/d_{\text{rot}}} \quad (k \in [1, m]_{\mathbb{Z}}). \quad (40)$$

## RoPE as a function from position to a matrix

Let the dimension be  $d_{\text{rot}} = 2m$  and choose frequencies

$$\theta_k := 10000^{-2(k-1)/d_{\text{rot}}} \quad (k \in [1, m]_{\mathbb{Z}}). \quad (40)$$

RoPE associates position  $i$  with the block-diagonal matrix

$$\mathbf{R}_i := \begin{bmatrix} \text{Rot}(i\theta_1) & & \mathbf{0} \\ & \ddots & \\ \mathbf{0} & & \text{Rot}(i\theta_m) \end{bmatrix} \in \mathbb{R}^{d_{\text{rot}}, d_{\text{rot}}}. \quad (41)$$

## RoPE as a function from position to a matrix

Let the dimension be  $d_{\text{rot}} = 2m$  and choose frequencies

$$\theta_k := 10000^{-2(k-1)/d_{\text{rot}}} \quad (k \in [1, m]_{\mathbb{Z}}). \quad (40)$$

RoPE associates position  $i$  with the block-diagonal matrix

$$\mathbf{R}_i := \begin{bmatrix} \text{Rot}(i\theta_1) & & \mathbf{0} \\ & \ddots & \\ \mathbf{0} & & \text{Rot}(i\theta_m) \end{bmatrix} \in \mathbb{R}^{d_{\text{rot}}, d_{\text{rot}}}. \quad (41)$$

Hence RoPE can be regarded as the function

$$i \longmapsto \mathbf{R}_i. \quad (42)$$

## RoPE converts absolute positions into relative positions

Consider arbitrary vectors  $\mathbf{v}^{(i)}, \mathbf{v}^{(j)} \in \mathbb{R}^{d_{\text{rot}}}$ , corresponding to position  $i$  and  $j$ .

Their RoPE-transformed inner product is

$$(\mathbf{R}_i \mathbf{v}^{(i)})^\top (\mathbf{R}_j \mathbf{v}^{(j)}) = (\mathbf{v}^{(i)})^\top \mathbf{R}_i^\top \mathbf{R}_j \mathbf{v}^{(j)} = (\mathbf{v}^{(i)})^\top \mathbf{R}_{j-i} \mathbf{v}^{(j)}. \quad (43)$$

## RoPE converts absolute positions into relative positions

Consider arbitrary vectors  $\mathbf{v}^{(i)}, \mathbf{v}^{(j)} \in \mathbb{R}^{d_{\text{rot}}}$ , corresponding to position  $i$  and  $j$ .

Their RoPE-transformed inner product is

$$(\mathbf{R}_i \mathbf{v}^{(i)})^\top (\mathbf{R}_j \mathbf{v}) = (\mathbf{v}^{(i)})^\top \mathbf{R}_i^\top \mathbf{R}_j \mathbf{v}^{(j)} = (\mathbf{v}^{(i)})^\top \mathbf{R}_{j-i} \mathbf{v}^{(j)}. \quad (43)$$

Thus,  $(\mathbf{R}_i \mathbf{u})^\top (\mathbf{R}_j \mathbf{v})$  depends on the positions  $i$  and  $j$  only through their **relative displacement**  $j - i$ .

## RoPE converts absolute positions into relative positions

For each two-dimensional block,

$$\text{Rot}(i\theta_k)^\top \text{Rot}(j\theta_k) = \text{Rot}((j - i)\theta_k). \quad (44)$$

## RoPE converts absolute positions into relative positions

For each two-dimensional block,

$$\text{Rot}(i\theta_k)^\top \text{Rot}(j\theta_k) = \text{Rot}((j-i)\theta_k). \quad (44)$$

Therefore,

$$\mathbf{R}_i^\top \mathbf{R}_j = \begin{bmatrix} \text{Rot}((j-i)\theta_1) & & \mathbf{0} \\ & \ddots & \\ \mathbf{0} & & \text{Rot}((j-i)\theta_m) \end{bmatrix} = \mathbf{R}_{j-i}. \quad (45)$$

## Positional encoding - answer

**Question:** How can we input order information into a Transformer?

## Positional encoding - answer

**Question:** How can we input order information into a Transformer?

**Answer:** We append the position index to each token. The NN converts each position index into a vector or matrix.

# Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can a neural network represent the diversity of words? - Embedding
4. Can we use representations in a NN? - Distance and similarities
- 5. How can we input order information? - Positional encoding**
6. Summary

# Outline

1. Introduction
2. Preliminaries: Mathematical Notations
3. How can a neural network represent the diversity of words? - Embedding
4. Can we use representations in a NN? - Distance and similarities
5. How can we input order information? - Positional encoding

## **6. Summary**

# Summary

Let's summarize the key takeaways from today's lecture.

- An **embedding layer** is a fully-connected layer that takes a one-hot vector as input. This is equivalent to a lookup operation, where the  $i$ -th token is mapped to the  $i$ -th column vector  $w_i$  of the layer's weight matrix.

# Summary

Let's summarize the key takeaways from today's lecture.

- An **embedding layer** is a fully-connected layer that takes a one-hot vector as input. This is equivalent to a lookup operation, where the  $i$ -th token is mapped to the  $i$ -th column vector  $w_i$  of the layer's weight matrix.
- This vector  $w_i$  is called the **representation** or **embedding** of token  $i$ . It's a dense, continuous vector in a lower-dimensional space.

# Summary

Let's summarize the key takeaways from today's lecture.

- An **embedding layer** is a fully-connected layer that takes a one-hot vector as input. This is equivalent to a lookup operation, where the  $i$ -th token is mapped to the  $i$ -th column vector  $w_i$  of the layer's weight matrix.
- This vector  $w_i$  is called the **representation** or **embedding** of token  $i$ . It's a dense, continuous vector in a lower-dimensional space.
- We defined three key measures to quantify relationships between embeddings: **Euclidean distance**, **standard inner product**, and **cosine similarity**. These allow us to capture the notion of "similarity" in the embedding space.

## References i

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville.  
**Deep Learning.**  
MIT Press, Cambridge, MA, 2016.
- [2] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean.  
**Efficient estimation of word representations in vector space.**  
In Proceedings of Workshop at ICLR 2013, 2013.
- [3] Jeffrey Pennington, Richard Socher, and Christopher D. Manning.  
**Glove: Global vectors for word representation.**  
In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 1532–1543, 2014.

## References ii

- [4] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu.

**Roformer: Enhanced transformer with rotary position embedding.**

Neurocomputing, 568:127063, 2024.

- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin.

**Attention is all you need.**

In Proceedings of NeurIPS 2017, 2017.